Chapter 10: Elementary Data Structures (CORMEN)

1) → A set is called dynamic if it can change over time. A dynamic set that allows you to insert elements into, delete elements from and test membership is called a dictionary.

→ Operations on dynamic sets

In the following S represents a set, K represents a key and x represents a pointer to an element in S:

(i) Search (S, K): returns a pointer x to an element in S such that key[x] = K or NIL if no such element belongs to S.

(ii) INSERT (S, x): inserts element pointed by x to S

(iii) DELETE (S, x)

(iv) MINIMUM (S): returns a pointer to the minimum element of S

(v) MAXIMUM (S): " " " " " maximum " " "

(vi) SUCCESSOR (S, x): given an element x, it returns a pointer to the next larger element in S, or NIL.

(vii) PREDECESSOR (S, x): returns a pointer to the element smaller than x or NIL.

(2) Stacks

A stack is a last-in, first-out (LIFO) list. We can implement a stack of ~~n~~ n elements with an array S[1..n]. The stack consists of elements S[1..top[S]], where S[1] is the element at the bottom of the stack and S[top[S]] is the element at the top.

When top[S] = 0, the stack contains no elements and is empty. top[S] indexes the most recently inserted element. If an empty stack is popped, we say the stack underflows and if top[S] exceeds n, the stack overflows.

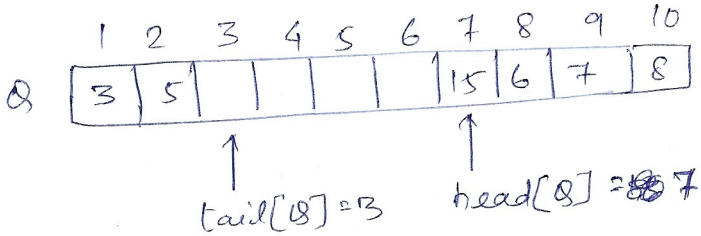
```
STACK-EMPTY(S)
1  if  top[S] = 0
2        then return TRUE
3        else return FALSE

PUSH(S, x)
1   top[S] ← top[S] + 1        ▷ should check for overflow here
2   S[top[S]] ← x

POP(S)
1  if STACK-EMPTY(S)
2       then error "underflow"
3       else top[S] ← top[S] - 1
4            return S[top[S] + 1]
```

(3) Queues

A queue is a first-in, first-out (FIFO) list. When an element is enqueued it is put at the tail of the queue. The "dequeued" element is removed from the head of the queue.



A queue of $n-1$ elements can be implemented using an array of n elements $Q[1..n]$. head[Q] indexes the element at the head of the queue. tail[Q] indexes the location where a newly arriving element would be put. The elements in the queue are at locations head[Q], head[Q]+1, ..., tail[Q]-1. The array indices wrap around, in the sense that index 1 immediately follows index n in a circular order.

When head[Q] = tail[Q], the queue is empty. When head[Q] = tail[Q] + 1 the queue is full.